

Задание 2 Airflow + Yandex Data Processing

Цель

Автоматизировать обработку файла через Apache Airflow и Yandex Data Processing.

По заданию входной файл должен быть CSV или Parquet и иметь размер не менее 50 МБ.

Подготовка данных

Для задания был подготовлен CSV-файл applications_v2.csv.

Создаем генератор csv на 50+ мб

```
import csv
import random
import argparse
from datetime import datetime, timedelta

def generate_applications(rows: int, output_path: str) -> None:
    regions = [
        "DE-HE", "DE-BE", "DE-BY", "DE-NW", "DE-HH",
        "FR-IDF", "FR-ARA", "IT-LOM", "ES-MD", "NL-NH"
    ]
    product_types = [
        "cash_loan",
        "credit_card",
        "mortgage",
        "car_loan",
        "consumer_loan",
        "refinancing"
    ]
    risk_levels = ["low", "medium", "high"]
    decision_statuses = ["approved", "rejected", "manual_review"]
    channels = ["mobile", "web", "office", "call_center"]
    start_date = datetime(2026, 5, 1, 8, 0, 0)
    with open(output_path, "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow([
            "application_id",
            "event_time",
            "customer_id",
            "region_code",
            "product_type",
            "requested_amount",
            "term_months",
            "credit_score",
            "risk_level",
            "decision_status",
            "approved_amount",
            "channel",
            "employee_review_flag",
            "processing_time_sec"
        ])
        for i in range(1, rows + 1):
            event_time = start_date + timedelta(
                seconds=random.randint(0, 31 * 24 * 60 * 60)
            )
            requested_amount = random.randint(1000, 50000)
            credit_score = random.randint(300, 850)
```

```

risk_level = random.choice(risk_levels)
if credit_score >= 700 and risk_level == "low":
    decision_status = "approved"
elif credit_score < 500 or risk_level == "high":
    decision_status = random.choice(["rejected", "manual_review"])
else:
    decision_status = random.choice(decision_statuses)
approved_amount = requested_amount if decision_status == "approved" else 0
employee_review_flag = decision_status == "manual_review"
writer.writerow([
    f'app_202605_{i:09d}',
    event_time.strftime("%Y-%m-%d %H:%M:%S"),
    f'cust_{random.randint(10000, 999999)}',
    random.choice(regions),
    random.choice(product_types),
    requested_amount,
    random.choice([6, 12, 18, 24, 36, 48, 60]),
    credit_score,
    risk_level,
    decision_status,
    approved_amount,
    random.choice(channels),
    str(employee_review_flag).lower(),
    random.randint(5, 300)
])
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--rows", type=int, default=500000)
    parser.add_argument("--out", type=str, default="task_2/data/applications_v2.csv")
    args = parser.parse_args()
    generate_applications(args.rows, args.out)
    print(f"Файл создан: {args.out}")
    print(f"Количество строк: {args.rows}")

```

Файл содержит данные по заявкам клиентов:

application_id	идентификатор заявки
event_time	дата и время события
customer_id	идентификатор клиента
region_code	код региона
product_type	тип продукта
requested_amount	запрошенная сумма
term_months	срок кредита
credit_score	кредитный рейтинг клиента
risk_level	уровень риска
decision_status	статус решения
approved_amount	одобренная сумма
channel	канал обращения
employee_review_flag	признак ручной проверки
processing_time_sec	время обработки заявки

Количество строк в файле: 500000.

Размер файла: 57.10 МБ.

Генерируем Файл объемом более 50 мб и выполняем проверку первых сгенерированных строк.

В моем случае объем 57.1 мб с 50000 строками

```
PS C:\Git\Kondratjev_Vladislav_ETL_Module4_Exam> code task_2\scripts\generate_applications_v2.py
PS C:\Git\Kondratjev_Vladislav_ETL_Module4_Exam> python task_2\scripts\generate_applications_v2.py --rows 50000 --out t
ask_2\data\applications_v2.csv
Файл создан: task_2\data\applications_v2.csv
Количество строк: 50000
PS C:\Git\Kondratjev_Vladislav_ETL_Module4_Exam> (Get-Item task_2\data\applications_v2.csv).Length / 1MB
57.0992269515991
PS C:\Git\Kondratjev_Vladislav_ETL_Module4_Exam> Get-Content task_2\data\applications_v2.csv -TotalCount 5
application_id,event_time,customer_id,region_code,product_type,requested_amount,term_months,credit_score,risk_level,dec
ision_status,approved_amount,channel,employee_review_flag,processing_time_sec
app_202605_000000001,2026-05-20 07:28:49,cust_791357,FR-ARA,car_loan,37678,6,654,medium,rejected,0,call_center,false,149
app_202605_000000002,2026-05-04 11:45:42,cust_264548,DE-BY,refinancing,5854,18,515,medium,rejected,0,call_center,false,2
54
app_202605_000000003,2026-05-27 18:07:19,cust_980986,DE-BY,cash_loan,37422,12,614,low,rejected,0,office,false,68
app_202605_000000004,2026-05-31 11:18:03,cust_110246,DE-HE,refinancing,45277,6,811,high,rejected,0,web,false,73
PS C:\Git\Kondratjev_Vladislav_ETL_Module4_Exam>
```

Файл был загружен в Object Storage по пути:

task_2/input/applications_v2.csv

PySpark-обработка

Для обработки данных был подготовлен PySpark-скрипт:

task_2/pyspark/process_applications.py

Скрипт выполняет следующие действия:

1. Читает CSV-файл из Object Storage.
2. Приводит числовые поля к нужным типам.
3. Добавляет поле event_date на основе event_time.
4. Группирует данные по:
 1. event_date
 2. region_code
 3. product_type
 4. decision_status
5. Рассчитывает агрегированные показатели:
 1. количество заявок;
 2. сумму запрошенных средств;
 3. сумму одобренных средств;
 4. средний кредитный рейтинг;
 5. среднее время обработки заявки.
6. Сохраняет результат в формате Parquet.

Результат сохраняется в Object Storage по пути:

task_2/output/applications_summary/

Создаем PySpark задание (файл process_applications.py)

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
import argparse
```

```

def main(input_path: str, output_path: str) -> None:
    spark = (
        SparkSession.builder
        .appName("task2_applications_processing")
        .getOrCreate()
    )
    df = (
        spark.read
        .option("header", "true")
        .option("inferSchema", "true")
        .csv(input_path)
    )
    cleaned = (
        df
        .withColumn("requested_amount", F.col("requested_amount").cast("double"))
        .withColumn("approved_amount", F.col("approved_amount").cast("double"))
        .withColumn("credit_score", F.col("credit_score").cast("int"))
        .withColumn("processing_time_sec", F.col("processing_time_sec").cast("int"))
        .withColumn("event_date", F.to_date("event_time"))
    )
    summary = (
        cleaned
        .groupBy("event_date", "region_code", "product_type", "decision_status")
        .agg(
            F.count("*").alias("applications_cnt"),
            F.sum("requested_amount").alias("requested_amount_sum"),
            F.sum("approved_amount").alias("approved_amount_sum"),
            F.avg("credit_score").alias("avg_credit_score"),
            F.avg("processing_time_sec").alias("avg_processing_time_sec")
        )
        .orderBy("event_date", "region_code", "product_type", "decision_status")
    )
    summary.write.mode("overwrite").parquet(output_path)
    spark.stop()
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", required=True)
    parser.add_argument("--output", required=True)
    args = parser.parse_args()
    main(args.input, args.output)

```

PySpark-задание в Object Storage:

cloud-vmkondrateveduhsru default Object Storage / Бакеты / etl-exam-vladislav-transactions-20260615-01 / task_2 / input

etl-exam-vladislav-transactions-20260615-0172.15 MB / 1 TB

Объекты

Настройки

Безопасность

Мониторинг

Операции

Объекты

Поиск по началу имени объекта, включая префикс

Имя	Размер	Класс хранилища	Последнее изменение	Метки
applications_v2.csv	57.1 MB	Стандартное	16.06.2026, в 21:35	

cloud-vmkondrateveduhsru default Object Storage / Бакеты / etl-exam-vladislav-transactions-20260615-01 / task_2 / pyspark

etl-exam-vladislav-transactions-20260615-0172.15 MB / 1 TB

Объекты

Настройки

Безопасность

Мониторинг

Операции

Объекты

Поиск по началу имени объекта, включая префикс

Имя	Размер	Класс хранилища	Последнее изменение	Метки
process_applications.py	1.68 KB	Стандартное	16.06.2026, в 21:39	

cloud-vmkondrateveduhsru default Virtual Private Cloud / Группы безопасности / dataproc-sg

dataproc-sgГруппа безопасности

Обзор

Обзор

Общее

Идентификатор.....enpbkt213m71ogc6m7f9

Статус.....Active

Имя.....dataproc-sg

Сеть.....default

Правила

Входящий трафикИсходящий трафик

Протокол	Диапазон портов	Тип источника	Источник	Описание
Any	0-65535	Группа безопасности	Self	internal-self

cloud-vmkondrateveduhsru default Virtual Private Cloud / Группы безопасности / dataproc-sg

dataproc-sg
Группа безопасности

Обзор

Обзор

Общее

Идентификатор.....enpbkt213m71ogc6m7f9

Статус.....Active

Имя.....dataproc-sg

Сеть.....default

Правила

Входящий трафикИсходящий трафик

Протокол	Диапазон портов	Тип назначения	Назначение	Описание
Any	0-65535	Группа безопасности	Self	internal-self
TCP	443	CIDR	0.0.0.0/0	https-out
UDP	123	CIDR	0.0.0.0/0	https-out

cloud-vmkondrateveduhsru default Virtual Private Cloud / Облачные сети / default / default-ru-central1-b

default-ru-central1-b
Подсеть

Обзор

IP-Адреса

Операции

Подсеть

Общее

Идентификатор.....e21k2srnjm4v6c616s3p

Статус.....Active

Имя.....default-ru-central1-b

Описание.....Auto-created default subnet for zone ru-central1-b in default

Зона доступности.....ru-central1-b

Сеть.....default

IPv4 CIDR.....10.129.0.0/24

Таблица маршрутизации.....dataproc-route-table x

Изменение таблицы маршрутизации

Имя *

Описание

Метки

Статические маршруты

Префикс назначения: 0.0.0.0/0, Шлюз [dataproc-nat](#) X

 Переопределение маршрутов в интернет имеет ряд важных последствий для внешней связности всех ВМ в подсетях, к которым будет привязана эта таблица. Пожалуйста, [ознакомьтесь с ними](#).

Airflow DAG

Для автоматизации был создан DAG:

task_2/dags/task2_dataproc_airflow_dag.py

```
from datetime import datetime

from airflow import DAG
from airflow.utils.trigger_rule import TriggerRule

from airflow.providers.yandex.operators.yandexcloud_dataproc import (
    DataprocCreateClusterOperator,
    DataprocCreatePysparkJobOperator,
    DataprocDeleteClusterOperator,
)

FOLDER_ID = "b1gtbeuc1thn2lkp2k9q"
ZONE = "ru-central1-b"

BUCKET = "etl-exam-vladislav-transactions-20260615-01"

SUBNET_ID = "e2lk2srnjm4v6c616s3p"
SERVICE_ACCOUNT_ID = "aje41u1agre3fumu8qtr"
SECURITY_GROUP_ID = "enpbkt213m71ogc6m7f9"

SSH_PUBLIC_KEY = "ssh-ed25519"
```

```
AAAAC3NzaC1lZDI1NTE5AAAAIPmb7264ybsFUT16MXuGt93vNrf82c0r6fW3ex  
YSrOHA dataproc-airflow"
```

```
INPUT_PATH = f"s3a://{BUCKET}/task_2/input/applications_v2.csv"  
PYSPARK_SCRIPT = f"s3a://{BUCKET}/task_2/pyspark/process_applications.py"  
OUTPUT_PATH = f"s3a://{BUCKET}/task_2/output/applications_summary"
```

```
with DAG(  
    dag_id="task2_dataproc_airflow_dag",  
    start_date=datetime(2026, 6, 16),  
    schedule=None,  
    catchup=False,  
    tags=["exam", "task_2", "dataproc", "pyspark"],  
) as dag:  
    create_cluster = DataprocCreateClusterOperator(  
        task_id="create_dataproc_cluster",  
        folder_id=FOLDER_ID,  
        cluster_name="task2-dataproc-cluster",  
        cluster_description="Temporary cluster for ETL exam task 2",  
        cluster_image_version="2.1",  
        zone=ZONE,  
        subnet_id=SUBNET_ID,  
        service_account_id=SERVICE_ACCOUNT_ID,  
        security_group_ids=[SECURITY_GROUP_ID],  
        ssh_public_keys=[SSH_PUBLIC_KEY],  
        s3_bucket=BUCKET,  
        services=["HDFS", "YARN", "SPARK"],  
        masternode_resource_preset="s2.small",  
        masternode_disk_type="network-hdd",  
        masternode_disk_size=32,  
        datanode_count=1,  
        datanode_resource_preset="s2.small",  
        datanode_disk_type="network-hdd",  
        datanode_disk_size=32,  
        computenode_count=0,  
        properties={  
            "spark:spark.submit.deployMode": "cluster",  
            "dataproc:disable_cloud_logging": "false",  
        },  
    )  
    run_pyspark_job = DataprocCreatePysparkJobOperator(  
        task_id="run_pyspark_job",  
        cluster_id=create_cluster.output,  
        main_python_file_uri=PYSPARK_SCRIPT,  
        args=[  
            "--input",  
            INPUT_PATH,  
            "--output",  
            OUTPUT_PATH,  
        ],  
        properties={
```



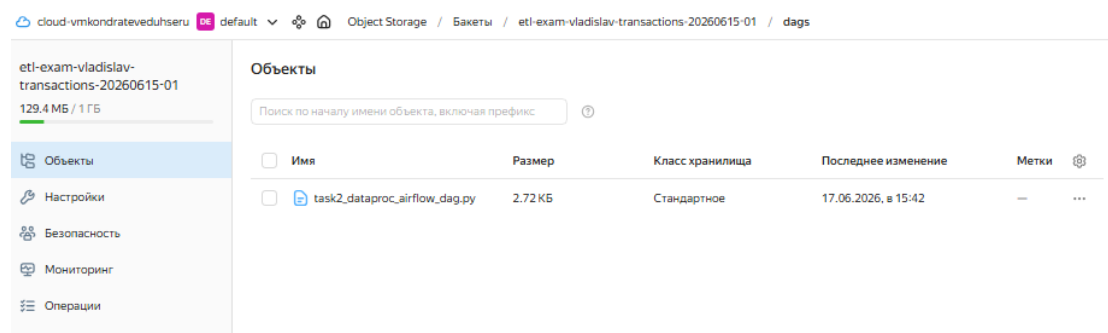
```

        "spark.submit.deployMode": "cluster",
    },
    name="task2-applications-processing",
)
delete_cluster = DataprocDeleteClusterOperator(
    task_id="delete_dataproc_cluster",
    cluster_id=create_cluster.output,
    trigger_rule=TriggerRule.ALL_DONE,
)
create_cluster >> run_pyspark_job >> delete_cluster

```

Также копия DAG была загружена в Object Storage в папку:

dags/task2_dataproc_airflow_dag.py



DAG состоит из трёх задач:

Задача	Описание
create_dataproc_cluster	создание временного кластера Yandex Data Processing
run_pyspark_job	запуск PySpark-скрипта на кластере
delete_dataproc_cluster	удаление кластера после выполнения обработки

Кластер создавался во временном режиме только на время выполнения задания.

Для кластера использовались сервисы:

- HDFS
- YARN
- SPARK

Запуск DAG

После загрузки DAG в Managed Service for Apache Airflow он был запущен вручную.

В ходе настройки были исправлены несколько проблем:

- добавлен публичный SSH-ключ для создаваемого кластера;
- указана версия образа Data Processing;
- исправлен идентификатор подсети;
- исправлен сервисный аккаунт кластера;
- набор сервисов кластера изменён на HDFS, YARN, SPARK.

После исправлений DAG выполнялся успешно.

Скриншот успешного выполнения DAG:



Результат

Задание выполнено.

Airflow успешно создал временный кластер Yandex Data Processing, запустил PySpark-обработку файла размером более 50 МБ, сохранил результат в Object Storage и удалил кластер после завершения работы.

